lecture 11

recall potential method

dynamic array: push elements to end of array
array needs to resize to increase capacity

reserving just the right amount: O(n) amortized push
reserving double the space: O(1) amortized push

Prove amortized time analysis

potential method is like induction:

recall induction: base case P(0)
inductive hypothesis: IH
inductive step: prove P(n) and IH → P(n+1)

potential method: look at special operations and prove run time.

Potential Method: Let S_0 be the initial state of your data structure, and $S_1, \dots S_n$ be the states after an arbitrary sequence of n operations.

Suppose you can prove:

1. $\Phi(S_i) \geq \Phi(S_0)$ potential at a state i has to be at least initial potential
2. for any i in n, if t_i is the time taken by operation i, then

$$t_i + (\Phi(S_i) - \Phi(S_{i-1})) \leq f(n)$$

Then, the amortized time per operation is $O(f(n))$

To prove amortized $O(f(n))$, you need

1. define $\Phi(S_i)$ such that it goes below initial potential
2. time per operation + change of potential can never be greater than $f(n)$

To prove $O(1)$, for a large operation t_i, the change in potential has to be a large negative value

---

Potential method Example

Growing Array:

Recall $O(1)$ time if not resizing $\rightarrow \leq 1 \cdot C$
$O(i)$ time if resizing with $i-1$ items $\rightarrow \leq n \cdot C$ time

Φ = 2 · C · [# of items in 2nd half of the array]

A B C $\Phi = 0$

A B C D $\Phi = 2 \cdot C \cdot 1$

check conditions
- first condition
  $I(S_0) = 0$. all $I(S_i) \geq 0$ ✓
- second condition
  case 1: no resize

  case 2: resize

  $t_i + (\Phi(S_i) - \Phi(S_{i-1})) \leq f(n)$

Φ = 2 · C · [# of items in 2nd half of the array]

Recall $O(1)$ time if not resizing $\rightarrow \leq 1 \cdot C$
$O(i)$ time if resizing with $i-1$ items $\rightarrow \leq n \cdot C$ time

$\Phi(S_0) = 2 \cdot C \cdot 0 = 0$ first condition ✓
$\Phi(S_i) \geq 0$

running time and change of potential for a normal (non-resizing) insertion

when it is not a resizing operation, it is at most on the right hand side adding an element



$t_i \leq 1 \cdot C$

$\Delta\Phi = 2 \cdot C \cdot (S_i) - 2 \cdot C \cdot (S_{i-1})$
$= 2 \cdot C$

$t_i + \Delta\Phi \leq 1C + 2C$
$\leq 3C$
$= O(1)$

---

running time and change of potential for a resizing insertion

Φ = 2 · C · [# of items in 2nd half of the array]

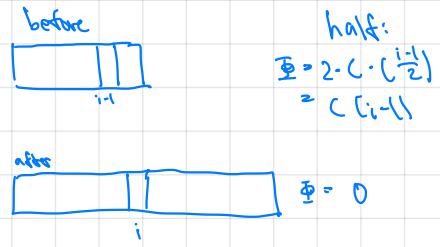
Recall $O(1)$ time if not resizing $\rightarrow \leq 1 \cdot C$
$O(i)$ time if resizing with $i-1$ items $\rightarrow \leq n \cdot C$ time

$t_i = i \cdot C$ after push, there are i elements

$\Delta\Phi = 0$
$= -C(i-1)$

if we resize, the array was full and $\frac{1}{2}$ was half full

$t_i + \Delta\Phi \leq i \cdot C - C(i-1)$
$\leq i \cdot C - i \cdot C + C$
$\leq C$
$= O(1)$



Thus, the amortized time per operation is $O(3C) = O(1)$

---

the potential function is summarizing what happens in the past

battery
- balance between time and potential function

will have to come up with one

fast operations increase potential
slow operations decrease potential

Potential Method Proof

Proof:

$T := \sum_{i}^{n} t_i$ time taken by all n operations

Want to show $T \leq n f(n)$ amortized time * n

$nf(n) = \sum_{i=1}^{n} f(n)$ telescope

$\geq \sum_{i=1}^{n} [t_i + (\Phi(s_i) - \Phi(s_{i-1}))]$

Given: $t_i + [\Phi(s_i) - \Phi(s_{i-1})] \leq f(n)$

$nf(n)$

$\geq (t_1 + \Phi(s_1) - \Phi(s_0)) + [t_2 + \Phi(s_2) - \Phi(s_1)] + (t_3 + \Phi(s_3) - \Phi(s_2)) \cdots (t_n + \Phi(s_n) - \Phi(s_{n-1}))$

$\geq t_1 + t_2 + \cdots + t_n + \Phi(s_n) - \Phi(s_0)$

$\geq T + \Phi(s_n) - \Phi(s_0)$

$\geq T + 0$

$\geq T$

Given

$\Phi(s_i) \geq \Phi(s_0)$

$\Phi(s_i) - \Phi(s_0) \geq 0$

$nf(n) \geq T$

Shows that the total time is less than $nf(n)$

Splay trees

- one of the most interesting balanced BST algorithms
- amortized $\log(n)$
- worst case is actually bad

- after you find a node in a tree, pull that node up the tree to make more helpful
- pull to root to make that work useful

def splay(node):

while node is not root:

if node is zig from root:

do splay-zig rotation to move node to the root level

if node is zig-zig from grandparent:

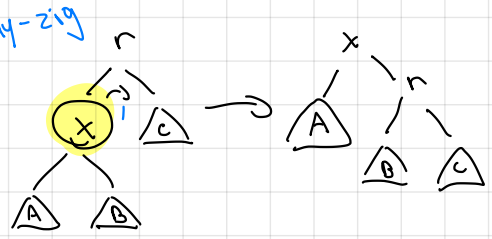
do splay-zig-zig rotation to move node up two levels

if node is zig-zag from grandparent:

do splay-zig-zag rotation to move node up two levels

3 cases:

Splay-zig



Steps

1. 1 rotation

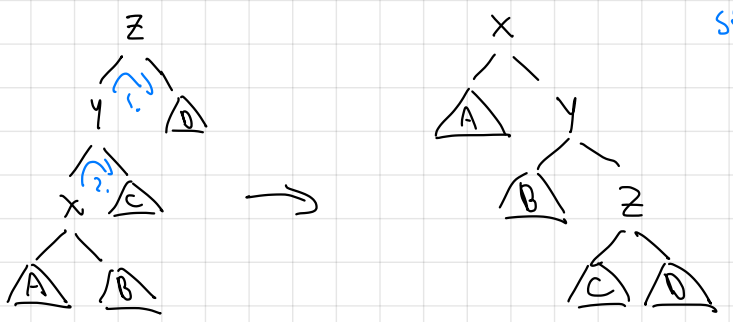
1 splay node that is 1 step off from the root

these operations:

- get amortized time bound

Splay zig-zig

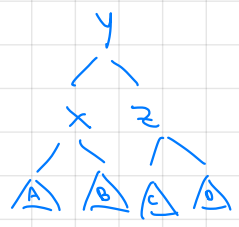
left-left
right-right from grandparent



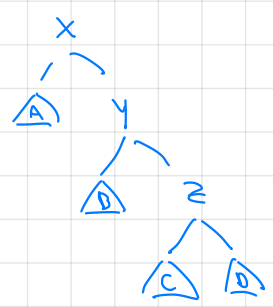
Steps

1. rotate around grandparent
2. rotate around parent

1.

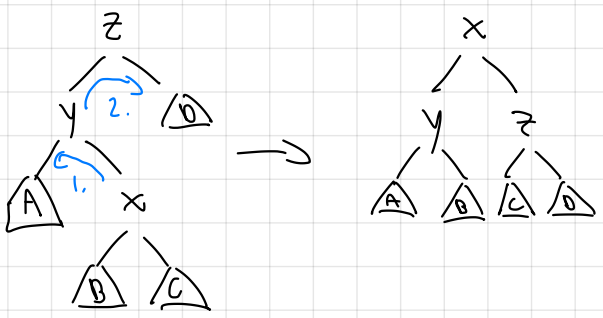


2.



Splay zig-zag

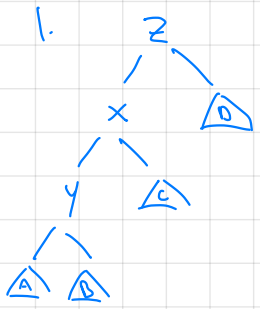
left right
right left from grandparent



Steps:

1. rotate around parent
2. rotate around grandparent

1.



2.

